

Guilherme Molari

Senior Software Engineer • Web, mobile & games

Londrina, PR, Brasil yguilz123@gmail.com github.com/gmolari
linkedin.com/in/guilherme-molari-5203a8236

Senior full stack engineer who builds scalable web applications, APIs and mobile apps with Next.js, React, TypeScript, Node.js and .NET. I have shipped high-traffic real-time platforms, payment and third-party integrations, back offices, reusable component libraries and performance work, always close to design in Figma. As Tech Lead at Ana Gaming I led a team of five front-end and back-end developers, set coding standards and ran code review and mentoring. I also build mobile apps (React Native, Kotlin, Java), web games with PixiJS, and use AI agents, Claude Code and MCP to automate development workflows.

EXPERIENCE

GMaster • **Full Stack Developer**

Evolved an established internal system for client management and financial consulting. Built REST endpoints in C# and .NET, new screens in Next.js and took part in architecture decisions, plus data modeling and queries, integrations, performance fixes, bug hunting and code review across the whole stack.

Ana Gaming • **Technical Lead**

Led a team of five front-end and back-end developers. Owned architecture decisions, delivery planning, coding standards, code review and technical mentoring.

StartBet • **Front-end Engineer**

Built gamification features and platform solutions for the betting product: reusable components, acquisition flows and tools that helped the team ship faster.

Multibet • **Front-end Engineer**

Built features across the product and internal tools: Multi-verso, the refer-a-friend back office, admin tools for popups and minigames, and landing pages, working from Figma with React Query and Zustand.

Supernova Gaming • **Senior Front-end Engineer**

One of the first engineers on the Supernova Bet platform: real-time features, REST and payment provider integrations, back offices and a shared component library. Later took

responsibility for the gamification layer together with the team and kicked off its projects. The gamification loop front-end I built 100% on my own.

PROJECTS

Supernova Bet

Supernova Gaming • Founding front-end engineer

Betting and casino platform front-end, built from the first commit. Casino lobby, sports, account and payment flows with real-time updates over Socket.IO, REST integrations with game and payment providers, and a shared component library.

Stack: Next.js, React, TypeScript, Socket.IO, Zustand, React Query

Gamification Loop

Supernova Gaming • Play4Tune • Front-end 100% solo • initiative led with the team

The retention layer of an iGaming platform: missions, levels, rewards and minigames living inside the product. Players progress through missions and levels and earn rewards they can spend or play with. Minigames and UI come from the PGX UI library, UI state lives in Zustand and server state in React Query, and everything is configured from the gamification back office.

Stack: Next.js, React, TypeScript, PixiJS, Framer Motion, Zustand, React Query

Gamification Back Office

Supernova Gaming • Play4Tune • Built entirely solo

The admin panel that runs the whole gamification system: operators create and tune every mission, reward and game without a deploy. Missions and quests, store, points, levels, VIP ranks, tournaments, prediction pools, minigames, jackpots, themes, onboarding and webhooks, all multi-currency and multi-language, plus a Player 360 view and live dashboards. Builders work by drag and drop with a preview of the real app screen. Security first: two-factor authentication (TOTP with QR code and recovery codes), sessions in signed httpOnly cookies so the token never reaches the browser, server-side token injection and sanitized rich text. A super admin console provisions new tenants, and the panel adapts to each tenant's system version.

Stack: Next.js, React, TypeScript, React Query, TanStack Table, React Hook Form, Zod, Recharts, PixiJS, next-intl, PGX UI

PGX UI

Supernova Gaming • Play4Tune • All the code solo • designs by the UX team in Figma

The company's component library: 81 React components, dynamic theming and seven minigames, shared by the gamification loop, the back office and Supernova Bet. Each brand's theme arrives from the API as tokens and flows through CSS variables into Tailwind, so one library dresses every white-label site. I wrote all the code, turning the UX team's Figma components into React, mapped back to Figma with Code Connect and documented in Storybook. The minigame engine runs on PixiJS 8 with the draw sealed on the server: wheel, mystery box and scratch card in production, plus mines, crash, penalty and slot ready on the same contract, with per-instance sound and assets uploaded from the back office.

Stack: React, TypeScript, Tailwind, PixiJS, Framer Motion, Storybook, Figma, Vitest, npm

VortexVolley

Personal project • Solo, full stack

A volleyball team manager: mobile app, API and a WhatsApp bot. Expo app with real-time updates over Socket.IO, a NestJS API with JWT access and refresh tokens and roles, PostgreSQL through Prisma, and a WhatsApp bot that syncs the game list with the group.

Stack: Expo, React Native, NestJS, Prisma, PostgreSQL, Socket.IO

↗ Source code

Bigette Fotos e Vídeos

Freelance • Solo, front and back end

Landing page for a photographer in Londrina, with its own admin back end. Next.js 16 and React 19 with 3D scenes that only load on capable devices. Behind it, an authenticated admin panel (JWT and bcrypt) with PostgreSQL and Drizzle, media uploads to Vercel Blob and drag-and-drop gallery ordering. Full SEO setup.

Stack: Next.js, React, Tailwind, Three.js, Drizzle, PostgreSQL, Vercel Blob

↗ Source code

Ad Monetization Engine

R&D lab • casual mobile games • Solo: engine and three games

An ad monetization engine for casual mobile games. For every player and moment it decides whether to show a rewarded ad, an interstitial or a banner, so the studio earns from ads without burning its players. Tuned like a product, not code: frequency caps, a grace period for new players, interstitials only at game over, rewarded revives, presets (recommended, aggressive, no ads), remote config and a live log that explains every decision. AdMob with automatic fallback to cross-promotion house ads, an ad-free purchase, consent (UMP and LGPD) and protections for minors come built in.

Stack: TypeScript, PixiJS v8, Vite, Capacitor, AdMob

Sweepstakes

R&D lab • Research and prototype

Market and legal research on sweepstakes casinos plus an original sweepstakes casino prototype. Eight original slot games on one parametric reel engine in WebGL, dual currency, free spins, VIP tiers, season pass, daily wheel, missions, responsible-gaming tools and a simulated geoblock and KYC flow. All art is drawn in code and runs on web and Android.

Stack: JavaScript, PixiJS, Canvas, Android WebView

CityBuilder

Personal project • Solo

A city-builder and RTS mod for Minecraft 1.20. Built on Fabric in Kotlin, extending the MineFortress mod with city-building and strategy mechanics.

Stack: Kotlin, Fabric, Java

⚡ SKILLS

TypeScript • JavaScript • React • Next.js • React Native • Expo • PixiJS • Three.js • Framer Motion • Tailwind CSS • Zustand • React Query • Node.js • NestJS • Socket.IO • C# • ASP.NET • Entity Framework • Kotlin • Java • PostgreSQL • MySQL • MongoDB • Prisma • Drizzle • Capacitor • Figma • Claude Code • MCP